

Metro Ticket Generating System in ServiceNow

ServiceNow Service Catalog Implementation

Project Documentation

Prepared for: ServiceNow PDI (Australia Instance)

Date: July 2026

1. Introduction

Project Title

Metro Ticket Booking & Smart Card Recharge System

Submitted By

Shaik Bushra Tabasum

Role: ServiceNow Developer (Individual Project) — responsible for end-to-end design, catalog build, scripting, testing, and documentation.

2. Project Overview

The **Metro Ticket Generating System** is a custom ServiceNow solution designed to digitize and automate the transit booking process for metro commuters. Built natively on the Now Platform, the application leverages the **Service Portal**, **Service Catalog**, **Flow Designer**, and **Notifications** to replace manual paper ticketing with dynamic digital tickets containing scannable QR codes.

1. Problem Statement & Solution

- **The Problem:** Traditional ticketing systems lead to long queuing times, high physical infrastructure costs, manual errors in fare calculations, and poor real-time operational visibility.
 - **The Solution:** A centralized Service Portal catalog item that allows passengers to select travel stations, automatically computes total fares using backend reference matrices, generates a digital ticket with a unique QR code, and instantly notifies users via automated email notifications.
-

2. Project Objectives

- **Automation:** Eliminate manual ticket issuance by automating fare generation and ticket record creation.
- **Self-Service:** Provide a clean, user-friendly Service Portal UI for passengers to easily select source, destination, journey type, and passenger count.

- **Instant Verification:** Deliver digital QR codes via email and portal records for seamless gate entry/exit verification.
 - **Operational Efficiency:** Enable real-time ticket lifecycle tracking (e.g., Active, Expired, Used) and automated ticket cleanup using Scheduled Jobs.
-

3. Key Target Personas & Roles

Persona / Role	Target User	Key Responsibilities / Usage
End User / Commuter	Metro Passengers	Submits catalog requests to book tickets, views ticket details, and receives QR codes via email.
Metro Admin	System Administrators	Manages station master data (u_metro_stations) and fare rules (u_fare_matrix).
ServiceNow Developer	Dev Team	Configures tables, Script Includes, Client Scripts, Flow Designer, and Notifications.

4. High-Level Architecture & Tech Stack

- **Platform Environment:** ServiceNow Personal Developer Instance (PDI) / Enterprise Instance.
- **Frontend Components:** Service Portal, Catalog Items, Client Scripts, Catalog UI Policies.
- **Backend Processing:** Custom Data Tables (u_metro_stations, u_metro_database_table, u_metro_tickets_table), Script Includes, Business Rules.

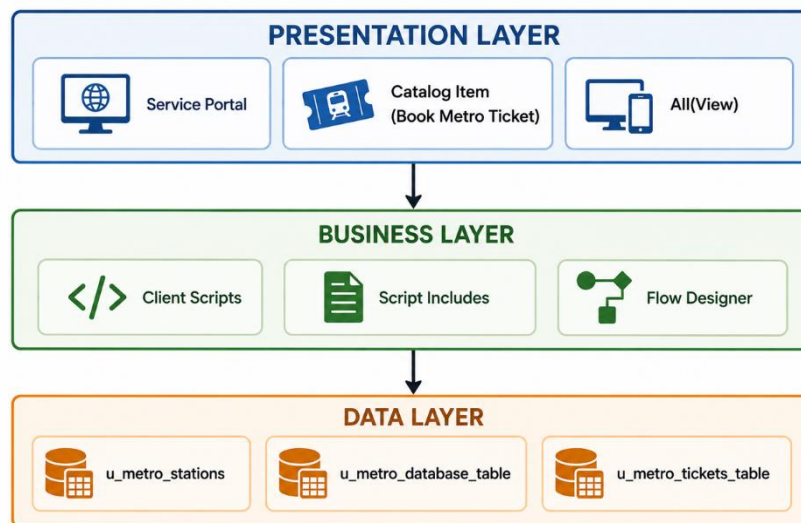
- **Automation Engine:** Flow Designer (RITM lifecycle management & email trigger) and Scheduled Jobs (Auto-expiry handling).

Features

- Single catalog item offering two branching services: Book a Metro Ticket and Recharge Smart Card
- Dynamic form fields shown/hidden based on the selected service using UI Policies
- Journey details capture: Starting Staation(starting from), Destination(Going to), Number of Passengers, Journey Type (Single/Return)
- Automatic fare calculation based on number of passengers and journey type via a Client Script
- Multiple payment modes (UPI, Net Banking, Credit/Debit Card, Others) with a conditional free-text field for 'Others'
- Smart card recharge capture: Smart Card Number, Smart Card Name, Recharge Amount
- QR code generation for the recharged smart card, displayed to the user on submission
- Automatic Requested Item (RITM) and Request (REQ) generation with full request tracking
- Additional Details view on the Requested Item form showing all submitted values

3. Architecture

The solution uses a **3-Tier Architecture** native to the Now Platform:



A. Presentation Layer (UI/UX)

- **Service Portal / Catalog Item:** The customer-facing interface where passengers input trip details.

- **Catalog UI Policies & Client Scripts:** Handles client-side validations, field visibility, and dynamic client-side calculations.

B. Business Logic Layer

- **Script Include (MetroTicketUtils):** Reusable server-side functions that fetch fare pricing from the matrix and handle ticket string logic.
- **Flow Designer:** Manages post-submission automation—updates request item status, generates email records, and sends notifications with attached QR code elements.

C. Data Layer (Custom Data Model)

- **Station Table (u_metro_stations):** Master list containing station names, zone codes, and active statuses.
- **Data Base Table(u_metro_database_table):** Stores the data, in this table to when flow triggered , the details are stored in database table
- **Ticket Booking Table (u_metro_tickets):** Transactional table storing generated ticket details, passenger count, total fare, purchase date/time, QR code payload, and current ticket status (Active, Expired, Used).

4. Setup Instructions

Prerequisites

- A ServiceNow Personal Developer Instance (PDI) — Australia data residency instance used for this project
- Admin (or catalog_admin) role access
- Access to Service Catalog > Catalog Definitions module

Installation / Configuration Steps

1. Log in to the ServiceNow PDI as System Administrator.
2. Copy the instance domain, e.g. <https://dev190678.service-now.com>
3. Append /sp to the URL to access the Service Portal, e.g. <https://dev190678.service-now.com/sp>
4. Navigate to Service Catalog > Catalog Definitions > Maintain Items > New to create the catalog item 'Book A Metro Ticket'.

5. Add variables, UI Policies, and Catalog Client Scripts as detailed in Section 7 (API / Scripts) below.

5. Catalog Structure

Since this is a ServiceNow implementation rather than a traditional client/server codebase, the equivalent of a 'folder structure' is the catalog configuration structure:

Catalog Item

- Book A Metro Ticket — Category: Office > Services

Variables (12 total)

- what_do_you_want_to_do_today (Multiple Choice) — branching control
- starting_from, going_to (Reference/String)
- no_of_passengers (Integer)
- type_of_journey (Choice: Single Journey / Return Journey)
- total_amomunt
- mode_of_payment (Choice: UPI,Credit/Debit Card,Net Banking ,Others)
- enter_payment_mode (String, conditional on 'Others')
- enter_smart_card_number, enter_smart_card_name, recharge_amount

Catalog Client Scripts (3 total)

- Auto FareCalculation — onChange — auto-calculates fare based on passengers/journey type
- FieldValidation — onChange — validates form input
- QR Generation — onSubmit — generates and displays the QR code for smart card recharge

UI Policies

- Show ticket-booking fields when 'Book a Metro Ticket' is selected
- Show recharge fields when 'Recharge Smart Card' is selected
- Show 'Amount Including Return' when Journey Type = Return
- Show 'Enter payment mode' when Mode of Payment = Others

6. Running the Application

As a ServiceNow catalog item, there is no separate frontend/backend server to start. The application runs directly within the ServiceNow instance:

1. Open the Service Portal:

```
https://<your-instance>.service-now.com/mt
```

2. Search for "Book A Metro Ticket" in the catalog search bar.
3. Fill in the required details and click Order Now to submit.

7. API Documentation

External API Used

QR Server API — used to generate a scannable QR code image representing the recharged smart card.

```
GET https://api.qrserver.com/v1/create-qr-code/  
?size=250x250  
&data=<encoded card number, amount, and reference ID>
```

Called from the 'QR Generation' Catalog Client Script (onSubmit) when the user selects Recharge Smart Card.

Internal Platform APIs

Function	Description
<code>g_form.getValue()</code>	Reads the current value of a catalog variable
<code>g_form.setValue()</code>	Sets a catalog variable value (used for auto fare calculation)
<code>g_form.getUniqueValue()</code>	Retrieves the RITM sys_id used as a QR reference number

8. Authentication

Authentication and authorization are handled natively by the ServiceNow platform:

- Users authenticate via standard ServiceNow login (username/password or SSO, depending on instance configuration)
- Service Portal session is maintained via ServiceNow's native session cookies
- Catalog item visibility and submission rights are controlled through the item's Availability and Permission tabs (roles/ACLs)
- No custom token-based authentication was implemented, as the entire solution runs within the authenticated ServiceNow session

9. User Interface

Tables:

1. Metro Stations Details Table

ServiceNow Developers | Metro Stations Details | Table | Home Page - Metro Ticket | SkillWallet | Ask Gemini | School

dev379536.service-now.com/how/nav/ui/classic/params/target/sys_db_object.do%3Fsys_id%3D4b710f323bde8b10f8b1f10f23e45a71%2...

serviceNow All Favorites History Workspaces Admin Table - Metro Stations Details

Application: Global

Label: Metro Stations Details

Name: u_metro_stations_details

Remote Table:

Columns Controls Application Access

Table Columns for text Search

Dictionary Entries

Column label	Type	Reference	Max length	Mandatory	Default value	Display
Updates	Integer	(empty)	40	false		false
Updated by	String	(empty)	40	false		false
Updated	Date/Time	(empty)	40	false		false
Station Code	String	(empty)	50	false		false
Created by	String	(empty)	40	false		false
Sys ID	Sys ID (GUID)	(empty)	32	false		false
Created	Date/Time	(empty)	40	false		false
Station Name	String	(empty)	100	false		true
Base Fare	Decimal	(empty)	15	false		false
Insert a new row...						

List

Metro Stations Details Station Name Search

Actions on selected rows... New

Station Name	Station Code	Base Fare
Nagole	104	20
Raidurg	105	15
L B Nagar	102	20
Miyapur	103	25
Ameerpet	101	10

2. Metro Database Table

Table Metro Database Table

Application: Global

Label: Metro Database Table

Name: u_metro_database_table

Remote Table:

Columns Controls Application Access

Table Columns for text Search

Dictionary Entries

Column label	Type	Reference	Max length	Mandatory	Default value	Display
Going To	Reference	Metro Stations Details	32	false		false
Enter payment mode	String	(empty)	40	false		false
Created by	String	(empty)	40	false		false
Recharge Amount	Decimal	(empty)	40	false		false
Enter Smart Card Name	String	(empty)	40	false		false
Type of Journey	Choice	(empty)	40	false		false
Created	Date/Time	(empty)	40	false		false
No of Passengers	Integer	(empty)	40	false		false
Sys ID	Sys ID (GUID)	(empty)	32	false		false
Mode of Payment	Choice	(empty)	40	false		false
Updated by	String	(empty)	40	false		false
Enter Smart Card Number	String	(empty)	40	false		false
Updates	Integer	(empty)	40	false		false
Total Amount	Decimal	(empty)	40	false		false
Starting From	Reference	Metro Stations Details	32	false		false
Updated	Date/Time	(empty)	40	false		false
Insert a new row...						

Delete Update Delete All Records

Copy
Try it
Update
Edit in Catalog browser
Delete

Related Links
Item Diagnostic
Run Point Scan
[SN UHBI] Versions (97)

Variables (13)
Variable Sets
Catalog UI Policies (3)
Catalog Client Scripts (4)
Available For
Not Available For
Categories (1)
Catalogs (1)
Catalog Data Lookup Definitions
Related Articles
Related Catalog Items
Assigned Topics

Order
Search

Catalog Item = Book A Metro Ticket

☐	Short description	Variable set	Conditions	Reverse if false	On load	Inherit	Updated	Order
☐	Show ticket fields when Book QR Ticket selected	(empty)		true	true	false	2026-07-30 01:43:35	100
☐	Reset QR confirmation on revisit	(empty)		false	false	false	2026-07-30 01:42:04	100
☐	mode of payment ui policy	(empty)		true	true	false	2026-07-30 00:20:38	100

1 to 3 of 3

Show ticket when book qr ticket selected:

Catalog UI Policy
Show ticket fields when Book QR Ticket selected

Update
Delete

Catalog UI policies are similar to standard UI policies. Catalog UI policies dynamically change variables that are part of a catalog item or change how variable sets are handled. Policies can also be applied when the variables are present in a Requested Item or Catalog Task form. [More Info](#)

Applies to
A Catalog Item

Application
Global

* Catalog Item
Book A Metro Ticket

Active
☒

* Short description
Show ticket fields when Book QR Ticket selected

When to Apply
Script

Catalog UI policy actions are applied only if all the following conditions are met:
1. The catalog UI policy is Active
2. The items in the Conditions field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions
Add Filter Condition
Add OR Clause

what_do_you_want_to_do_today
is
Book QR Ticket
AND OR X

Applies on a Catalog item view
☒

Applies on Catalog Tasks
☐

Applies on Requested Items
☐

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form

On load
☒

Reverse the effects of the catalog UI policy actions when the Conditions evaluate to false

Reverse if false
☒

Update
Delete

Related Links

[SN UHBI] Versions (11)

Catalog UI Policy Actions
Order
Search

Actions on selected rows...
New

UI policy = Show ticket fields when Book QR Ticket selected

☐	Name	Read only	Mandatory	Visible	Order
☐	starting_from	Leave alone	True	True	100
☐	going_to	Leave alone	True	True	200
☐	type_of_journey	Leave alone	True	True	300
☐	no_of_passengers	Leave alone	True	True	400
☐	mode_of_payment	Leave alone	True	True	500
☐	total_amount	Leave alone	Leave alone	True	600
☐	enter_smart_card_name	Leave alone	Leave alone	False	700
☐	recharge_amount	Leave alone	Leave alone	False	800
☐	enter_smart_card_number	Leave alone	Leave alone	False	800

1 to 9 of 9

Catalog UI Policy
Reset QR confirmation on revisit

Catalog UI policies are similar to standard UI policies. Catalog UI policies dynamically change variables that are part of a catalog item or change how variable sets are handled. Policies can also be applied when the variables are present in a Requested Item or Catalog Task form. [More info](#)

Applies to: Application:

* Catalog item:

* Short description:

Active: ☒

When to Apply | **Script**

Catalog UI policy actions are applied only if all the following conditions are met:

1. The catalog UI policy is **Active**
2. The items in the **Conditions** field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions:

Applies on a Catalog item view: ☒

Applies on Catalog Tasks: ☐

Applies on Requested Items: ☐

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form: ☐

On load: ☐

Reverse the effects of the catalog UI policy actions when the Conditions evaluate to false: ☐

Reverse if false: ☐

* Short description: Reset QR confirmation on revisit

When to Apply | **Script**

If **Run scripts** is selected, a specified script will be executed when the UI Policy **Conditions** evaluate to true, and a second script will be executed when the conditions evaluate to false

Run scripts: ☒

Run scripts in UI type:

Execute if true: ☒

```

1 function onCondition() {
2   g_form.setValue('qr_confirmed', false);
3 }

```

Execute if false: ☐

```

1 function onCondition() {
2
3 }

```

Mode of payment ui policy:

Catalog UI Policy
mode of payment ui policy

Catalog UI policies are similar to standard UI policies. Catalog UI policies dynamically change variables that are part of a catalog item or change how variable sets are handled. Policies can also be applied when the variables are present in a Requested Item or Catalog Task form. [More info](#)

Applies to: Application:

* Catalog item:

* Short description:

Active: ☒

When to Apply | **Script**

Catalog UI policy actions are applied only if all the following conditions are met:

1. The catalog UI policy is **Active**
2. The items in the **Conditions** field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions:

Applies on a Catalog item view: ☒

Applies on Catalog Tasks: ☐

Applies on Requested Items: ☐

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form: ☐

On load: ☒

Reverse the effects of the catalog UI policy actions when the Conditions evaluate to false: ☐

Reverse if false: ☒

Related Links
[Run Print Scan](#)
[View UI Policy Versions \(3\)](#)

Catalog UI Policy Actions Order Actions on selected rows:

UI policy = mode of payment ui policy	Name	Read only	Mandatory	Visible	Order
☐	enter_payment_mode	Leave alone	True	True	100

1 to 1 of 1

Run Point Scan

[SN Units] Versions (97)

Variables (13)

Variable Sets

Catalog UI Policies (3)

Catalog Client Scripts (4)

Available For

Not Available For

Categories (1)

Catalogs (1)

Catalog Data Lookup Definitions

Related Articles

Related Catalog Items

Assigned Topics

for text

Search

Actions on selected rows...

New

Catalog Item = Book A Metro Ticket

☐	Q	Name	Active	Table	View	Type	Updated
☐		QrModelCleanUP	true			onLoad	2026-07-30 00:49:53
☐		Auto Populate Fare Values	true			onChange	2026-07-29 07:17:19
☐		field validation	true			onChange	2026-07-30 00:48:08
☐		QrGeneration	true			onSubmit	2026-07-30 01:38:51

<<

<

1

>

>>

to 4 of 4

Catalog Client Script

Auto populate fare value:

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "jside.script.block.client.globals" to false.

Name	Auto Populate Fare Values	Application	Global
Applies to	A Catalog Item	Type	onChange
Active	☒	Catalog Item	Book A Metro Ticket
UI Type	All	Variable name	type_of_journey
Applies on a Catalog Item view		☒	
Applies on Requested Items		☐	
Applies on Catalog Tasks		☐	

Script

```

1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
  
```

```
function onChange(control, oldValue, newValue, isLoading) {
```

```
    if (isLoading || newValue == '') {
```

```
        return;
```

```
    }
```

```
    var source = g_form.getValue('starting_from');
```

```
    var destination = g_form.getValue('going_to');
```

```
    var numPassengers = parseInt(g_form.getValue('no_of_passengers'), 10) || 1;
```

```
    // Helper multiplier: Single = 1x fare, Return = 2x fare
```

```
    var multiplier = (newValue == '2') ? 2 : 1;
```

```
    // Ameerpet to L B Nagar (Base fare: 50)
```

```

    if (source == "ac8b0f763b12cb10f8b1f10f23e45ae4" && destination ==
'5febcb63b12cb10f8b1f10f23e45a23') {

```

```

    g_form.setValue('total_amount', numPassengers * 50 * multiplier);

```

```

}

```

```

// Ameerpet to Miyapur (Base fare: 60)

```

```

    else if (source == "ac8b0f763b12cb10f8b1f10f23e45ae4" && destination ==
'8e4c47363b12cb10f8b1f10f23e45a1b') {

```

```

    g_form.setValue('total_amount', numPassengers * 60 * multiplier);

```

```

}

```

```

// Ameerpet to Nagole (Base fare: 55)

```

```

    else if (source == "ac8b0f763b12cb10f8b1f10f23e45ae4" && destination ==
'265cc7f63b12cb10f8b1f10f23e45a9a') {

```

```

    g_form.setValue('total_amount', numPassengers * 55 * multiplier);

```

```

}

```

```

// Ameerpet to Raidurg (Base fare: 40)

```

```

    else if (source == "ac8b0f763b12cb10f8b1f10f23e45ae4" && destination ==
'5c7c8bf63b12cb10f8b1f10f23e45a7b') {

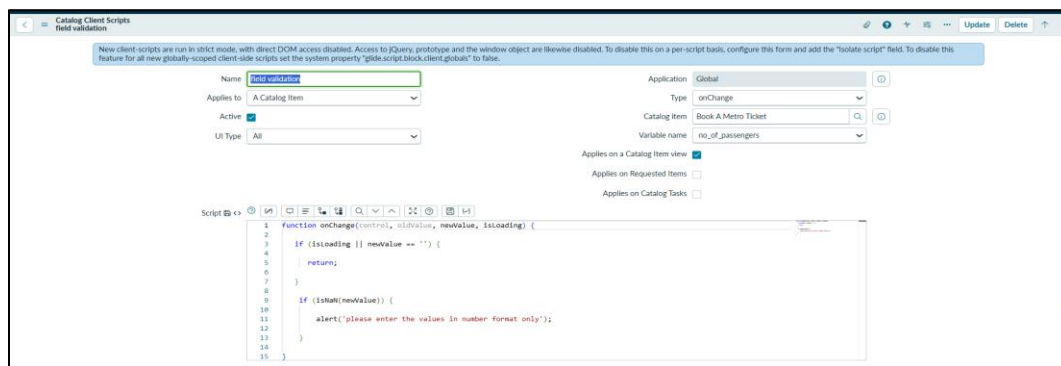
```

```

    g_form.setValue('total_amount', numPassengers * 40 * multiplier); } }

```

Filed validation:



QR Genration:

The screenshot shows the 'Catalog Client Scripts' configuration page for a script named 'QrGeneration'. At the top, there is a warning banner stating that the script is not supported due to the presence of certain variables. Below this, a detailed message explains that new client-scripts run in strict mode with disabled DOM access, and provides instructions on how to disable this feature. The configuration form includes fields for 'Name' (QrGeneration), 'Applies to' (A Catalog Item), 'Active' (checked), 'UI Type' (All), 'Application' (Global), 'Type' (onSubmit), and 'Catalog Item' (Book A Metro Ticket). There are also checkboxes for 'Applies on a Catalog Item view' (checked), 'Applies on Requested Items' (unchecked), and 'Applies on Catalog Tasks' (unchecked). At the bottom, a code editor shows the start of a function: `1 function onSubmit() {`.

```
function onSubmit() {
```

```
    var win = getSafeWindow();
```

```
    // Re-entrancy guard: prevents a second onSubmit firing from stacking
```

```
    // a second overlay while the first is still open/pending.
```

```
    if (win && win.__qrModalInProgress) {
```

```
        return false;
```

```
    } // Check BOTH the field value and a synchronous in-memory flag.
```

```
    var qrConfirmedVal = g_form.getValue('qr_confirmed');
```

```
    var qrConfirmedFlag = win && win.__qrConfirmedNow;
```

```
    if (qrConfirmedFlag || qrConfirmedVal === 'true' || qrConfirmedVal === true || qrConfirmedVal ===  
'1' || qrConfirmedVal === 1) {
```

```
        return true;
```

```
    }
```

```
    var actionType = g_form.getValue('what_do_you_want_to_do_today'); // Recharge Metro Card or  
    non-QR booking must submit normally without QR popup
```

```
    if (actionType !== 'book_qr_ticket') {
```

```

    return true;

}   var startStation = g_form.getDisplayValue('starting_from');

var endStation = g_form.getDisplayValue('going_to');

var passengers = g_form.getValue('no_of_passengers');

if (!startStation || !endStation) {

    return true;

}   // Build QR payload and generator URL

    var qrPayload = 'TYPE:Ticket|FROM:' + startStation + '|TO:' + endStation + '|PASSENGERS:' +
passengers;

    var qrImageUrl = 'https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=' +
encodeURIComponent(qrPayload);

if (typeof g_form.isVirtualAgentClient === 'function' && g_form.isVirtualAgentClient()) {

    g_form.addInfoMessage('QR Code generated! View here: ' + qrImageUrl);

    return true;

}   if (win) win.__qrModalInProgress = true;

// Defer modal creation so it renders cleanly over Service Portal overlays

setTimeout(function () {

    var shown = showQRModal(qrImageUrl);

    if (!shown) {

        if (win) win.__qrModalInProgress = false;

        g_form.addInfoMessage('QR Code generated! View here: ' + qrImageUrl);

    }

}

```

```

    }, 50);    return false;

}

function getSafeDocument() {

    try {

        if (typeof document !== 'undefined' && document && document.createElement) return document;

    } catch (e) {}

    try {

        if (typeof window !== 'undefined' && window && window.document &&
window.document.createElement) return window.document;

    } catch (e) {}

    try {

        if (typeof top !== 'undefined' && top && top.document && top.document.createElement) return
top.document;

    } catch (e) {}

    return null;

}

function getSafeWindow() {

    try {

        if (typeof window !== 'undefined' && window) return window;

    } catch (e) {}

    try {

        if (typeof top !== 'undefined' && top) return top;

    }

```



```

    } catch (e) {}

    try {

        if (typeof self !== 'undefined' && self) return self;

    } catch (e) {}

    return null;

}

function removeAllQrOverlays(doc) {

    if (!doc) return;

    var overlays = doc.querySelectorAll ? doc.querySelectorAll('#qrModalOverlay') : [];

    for (var i = 0; i < overlays.length; i++) {

        if (overlays[i] && overlays[i].parentNode) {

            overlays[i].parentNode.removeChild(overlays[i]);

        }

    }

}

function showQRModal(qrImageUrl) {

    var doc = getSafeDocument();

    if (!doc || !doc.body || typeof doc.createElement !== 'function') {

        return false;

    }

    removeAllQrOverlays(doc);

```

```

var overlay = doc.createElement('div');

overlay.id = 'qrModalOverlay';

overlay.style.cssText = 'position:fixed;top:0;left:0;width:100%;height:100%;' +

    'background:rgba(0,0,0,0.5);z-index:2147483647;display:flex;align-items:center;justify-
content:center;' +

    'pointer-events:auto;';

var modal = doc.createElement('div');

modal.style.cssText = 'background:#fff;border-radius:6px;width:320px;' +

    'box-shadow:0 4px 20px rgba(0,0,0,0.35);overflow:hidden;font-family:Arial,sans-serif;' +

    'position:relative;z-index:2147483647;pointer-events:auto;';

modal.innerHTML =

    '<div style="background:#123a5c;color:#fff;padding:10px 14px;display:flex;' +

        'justify-content:space-between;align-items:center;">' +

        '<span style="font-weight:bold;">Scan Your QR Here</span>' +

        '<span id="qrModalCloseX" style="cursor:pointer;font-size:18px;line-height:1;pointer-
events:auto;">&times;</span>' +

    '</div>' +

    '<div style="padding:20px;text-align:center;">' +

        '<img src="" + qrImageUrl + "" style="border:2px solid #c0392b;padding:8px;background:#fff;"
width="220" height="220"/>' +

    '</div>' +

    '<div style="padding:10px 14px;text-align:right;background:#f5f5f5;">' +

```

```
'<button type="button" id="qrModalCloseBtn" style="background:#123a5c;color:#fff;border:none;' +
```

```
'padding:6px 18px;border-radius:4px;cursor:pointer:pointer-events:auto;">Close</button>' +
```

```
'</div>';
```

```
overlay.appendChild(modal);
```

```
doc.body.appendChild(overlay);
```

```
var closeX = doc.getElementById('qrModalCloseX');
```

```
var closeBtn = doc.getElementById('qrModalCloseBtn');
```

```
if (closeX) {
```

```
    closeX.addEventListener('click', function (e) {
```

```
        e.preventDefault();
```

```
        e.stopPropagation();
```

```
        closeModalAndSubmit();
```

```
    }, true);
```

```
}
```

```
if (closeBtn) {
```

```
    closeBtn.addEventListener('click', function (e) {
```

```
        e.preventDefault();
```

```
        e.stopPropagation();
```

```
        closeModalAndSubmit();
```

```
    }, true);
```

```

}

overlay.addEventListener('click', function (e) {

    if (e.target === overlay) {

        closeModalAndSubmit();

    }

}, true);

return true;

}

function closeModalAndSubmit() {

    var win = getSafeWindow();

    var doc = getSafeDocument();

    removeAllQrOverlays(doc);

    g_form.setValue('qr_confirmed', 'true');

    if (win) win.__qrConfirmedNow = true;

    if (win) win.__qrModalInProgress = false;

    var orderBtn = doc ? doc.getElementById('oi_order_now_button') : null;

    if (orderBtn) {

        orderBtn.click();

    } else if (win && typeof win.processOrderNow === 'function') {

        win.processOrderNow(false);

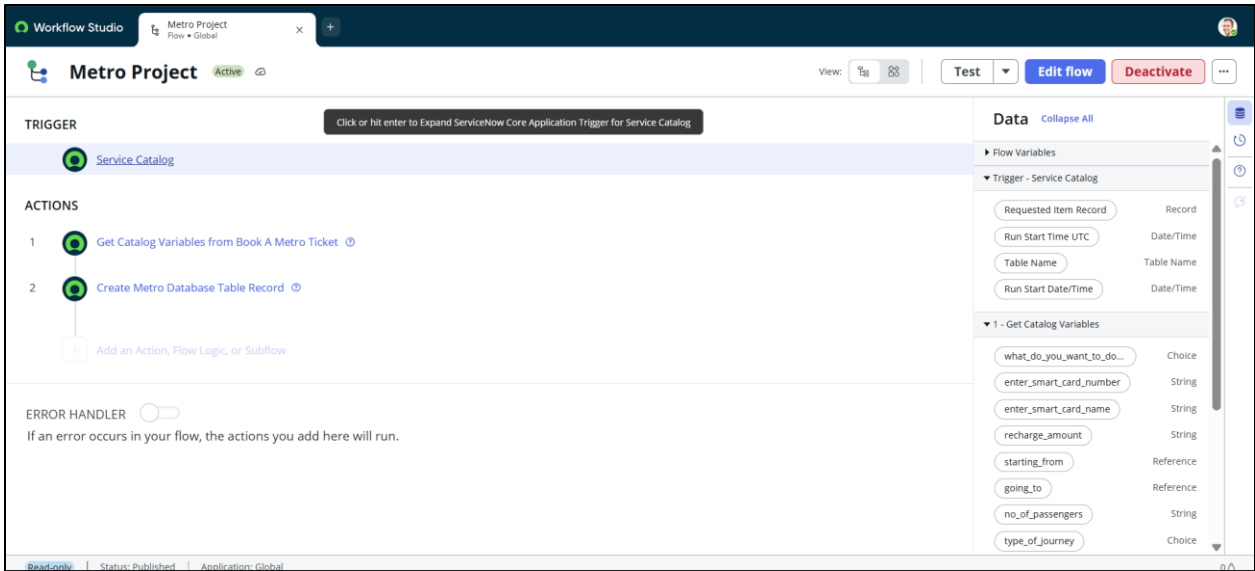
    } else {

```

```
g_form.addInfoMessage('Please click Order Now to complete your request.');
```

```
}}
```

Flow:



2 Create Metro Database Table Record

Action Properties

Action Create Record

Action Inputs

* Table

Enter Smart Card Name	X	1 - Get ... ▶ enter_smart_card...			
Enter Smart Card Number	X	1 - Get... ▶ enter_smart_card...			
Recharge Amount	X	1 - Get Catal... ▶ recharge_am...			
Starting From	X	1 - Get Catalog ... ▶ starting_fr...			
Going To	X	1 - Get Catalog Vari... ▶ going...			
* Fields					
No of Passengers	X	1 - Get Catal... ▶ no_of_passen...			
Type of Journey	X	1 - Get Catalo... ▶ type_of_jou...			
Total Amount	X	1 - Get Catalog ... ▶ total_amo...			
Mode of Payment	X	1 - Get Cata... ▶ mode_of_pay...			
+ Add field value					

Service Portal Home

← → ↺ dev379536.service-now.com/mt 🔍 ☆ 🏠

How can we help?

How can we help? 🔍

Request Something

Browse the catalog for services and items you need

Knowledge Base

Browse and search for articles, rate or submit feedback

Get Help

Contact support to make a request, or report a problem

Current Status

No system is reporting an issue

[More information...](#)

Top Rated Articles

My Assessments and Surveys

No assessments or surveys for you at the moment

Announcements

No information available

My Approvals

You have no pending approvals

My Open Incidents

Employee payroll application server is down.
INC0007001 • 3mo ago

Rain is leaking on main DNS Server

[Home](#) > [Search](#)

Sources

[All](#)
[Knowledge Bases](#)
[Catalogs](#)

FILTERS

All results for "metro"

[Book A Metro Ticket](#)

Metro

Ticket Booking System

Catalog Item — Initial View

[Home](#) > [Service Catalog](#) > [Office](#) > [Services](#) > [Book A Metro Ticket](#)

Book A Metro Ticket

Metro Ticket Booking System

Submit

* Indicates required

What do you want to do Today?

☒ Book QR Ticket
 ☐ Recharge Smart Card

* Starting From

* Going To

* No of Passengers

* Type of Journey

☐ -- None --
 ☒ Single Journey
 ☐ Return Journey

Total Amount

* Going To

* No of Passengers

* Type of Journey

☐ -- None --
 ☒ Single Journey
 ☐ Return Journey

Total Amount

* Mode of Payment

☐ UPI
 ☐ Credit/Debit Card
 ☐ Net Banking
 ☒ Other

* Enter payment mode

Submit

Required information

Enter payment mode

* Indicates required

What do you want to do Today?

☒ Book QR Ticket

☐ Recharge Smart Card

* Starting From

Ameerpet

* Going To

L B Nagar

* No of Passengers

2

* Type of Journey

☐ -- None --

☒ Single Journey

☐ Return Journey

Total Amount

100

* Mode of Payment

☒ UPI

☐ Credit/Debit Card

☐ Net Banking

☐ Other

Submit

Scan Your QR Here



Close

[Home](#) > Submitted Request

REQ0010008 - Metro Ticket Booking System

Type your message here...

Send



System Administrator

2026-07-30 22:20:39

REQ0010008 Created

Start

Requested Items

Metro Ticket Booking System

RITM0010008

☒ Request Approval (Pending - has not started)
 ☐ Assess or Scope Task (Pending - has not started)
 ☐ Provide Service (Pending - has not started)
 ☐ Completed (Pending - has not started)

Estimated completion 2026-08-01

Number	State
REQ0010008	Open
Priority	Created
4 - Low	just now
Price	Updated
\$0.00	just now

Requests are picked up within
4 hours (M-F 9-5)

Figure 2: Book A Metro Ticket catalog item with branching choice

Book a Metro Ticket — Journey Details

Your request has been submitted

Number
RITM0010009

State
Open

Created
just now

Updated
just now

Options

What do you want to do Today?
Book QR Ticket

Starting From
Amserpet

Going To
LB Nagar

No of Passengers
2

Type of Journey
Single Journey

Total Amount
100

Mode of Payment
UPI

QR Confirmed
Yes

Open Items

Back to Catalog
Continue Shopping
Home

Thank you, your request has been submitted

Order Number:
RITM0010009

Request Number:
RITM0010009 ⓘ

Approval Requested:
31-07-2026

Estimated Delivery:
05-08-2026

Description	Type of Journey	No. of Passengers	Total
A metro ticket between Metro Code 9424 to JBMSG is 50rs for adult in both the ways and available till 6:00 PM	Single	2	₹100
Total			₹100

Back to Catalog
Continue Shopping

Home

Figure 3: Journey fields revealed after selecting Book QR Ticket

Recharge Smart Card — Field View

The screenshot shows a web form titled "Book A Metro Ticket" with the subtitle "Metro Ticket Booking System". On the right side, there is a blue "Submit" button. The main form area contains the following elements:

- A heading "What do you want to do Today?" with two radio button options: "Book QR Ticket" (unselected) and "Recharge Smart Card" (selected).
- A text input field labeled "Enter Smart Card Number" containing the value "123456".
- A text input field labeled "Enter Smart Card Name" containing the value "Shaik".
- A text input field labeled "Recharge Amount" containing the value "400".

Figure 5: Recharge Smart Card fields alongside ticket fields (pre-fix)

Order Confirmation

The screenshot shows a modal dialog titled "Order Confirmation" with a close button (X) in the top right corner. The dialog contains the following elements:

- A label "Request for" followed by a dropdown menu showing "System Administrator".
- A checkbox labeled "Delivery Information (Optional)".
- A checkbox labeled "Special instructions (Optional)" followed by a large text input area.
- At the bottom right, there are two buttons: "Cancel" and "Submitting...".

Figure 6: Order Confirmation modal

Requested Item — Additional Details

Application

Global

ⓘ

Type

Multiple Choice

▼

Catalog Item

Book A Metro Ticket

🔍 ⓘ

Order

100

Active

☒

Mandatory

☒

Read only

☐

Hidden

☐

Disable automatic slot fill based on user context

☐

Question

Annotation

Type Specifications

Default Value

Auto-populate

Permission

Availability

* Default value

Copy

Update

Delete

Related Links

[In Point Scan](#)
[N Utils](#)
[Versions \(4\)](#)

☰

🔍

Question Choices

Order

▼

Search

⌂

🔍

📄

—

Actions on selected rows...

▼

New

Question = What do you want to do Today?

	Text	Value	Order	Inactive
	Recharge Smart Card	recharge_card	200	false
	Book a Metro Ticket		100	false

Figure 7: RITM generated with Additional Details tab

Your request has been submitted

Number

RITM0010010

Updated

just now

State

Open

Created

just now

Options

What do you want to do Today?	Enter Smart Card Number	Enter Smart Card Name	Recharge Amount	Mode of Payment
Recharge Smart Card	123456	Shaik	400	UPI

Metro database:

Metro Database Tables						
Starting From		Going To		Type of Journey		No of Passengers
Ameerpet		L B Nagar		Single Journey		2
						100
						UPI

10. Testing

Testing Strategy

Manual functional testing was performed directly on the ServiceNow PDI Service Portal, covering both branches of the catalog item.

Test Cases Executed

#	Test Case	Result
1	Book a Metro Ticket — Single Journey fare calculation	Pass
2	Book a Metro Ticket — Return Journey fare calculation	Pass
3	Payment Mode = Others reveals free-text field	Pass
4	Order submission generates RITM with correct Additional Details	Pass
5	Recharge Smart Card — UI Policy hides ticket fields	Pass
6	QR code generation on Recharge Smart Card submission	Pass

11. Screenshots / Demo

All key screens are captured in Section 9 (User Interface) above, covering the full navigation flow from Service Portal home through to Order Confirmation and Request Summary.

12. Known Issues

Resolved During Development

- UI Policy for 'Recharge Smart Card' did not hide the ticket-booking fields — root cause was a mismatch between the UI Policy condition and the actual Question Choice Value; corrected by verifying the exact Value column in Question Choices.
- QR popup showed 'Page not found' — root cause was use of GlideModal referencing a non-existent UI Page (external_qr_api_page); this API is Classic UI-only and is not available in Service Portal.
- QR Generation script threw 'ReferenceError: GlideModal is not defined' in the Service Portal — resolved by replacing GlideModal with a native window.open() popup rendering the QR Server API image directly.

Outstanding

- Browser popup blockers may prevent the QR window from opening on first attempt — requires the user to allow popups for the instance domain.
- QR Server API is an external third-party domain; outbound access must be confirmed on restricted/enterprise instances.

13. Future Enhancements

- Replace the third-party QR Server API with an in-platform QR generation library to remove the external dependency
- Display the QR code inline within the Requested Item's Additional Details view (via a Service Portal widget) instead of a separate popup window
- Add Flow Designer automation to route Recharge Smart Card requests to a fulfillment/approval flow distinct from ticket bookings
- Add mandatory attachment validation (e.g., payment proof) only when Mode of Payment is UPI or Others
- Add real fare-matrix lookup (via a Data Lookup Definition) instead of a flat per-passenger rate